

PROJECT DOCUMENTATION REPORT

ON

**EMPLOYEE RAISE ISSUE
MANAGEMENT**

Record Producer & Employee Center Integration

**A ServiceNow Application for Centralized Employee Issue
Submission**

**Submitted by
PANDLA SRILAKSHMI MOUNIKA
August 2026**

TABLE OF CONTENTS

1.SolutionContex	3
1.1 Business Contex	3
1.2 Existing Challenge	3
1.3 Solution Approach	4
2. Project Goals	5
3. Functional Highlights	6
4. Development Environment	7
4.1 Platform Access and Configuration.....	7
4.2 Development Instance	8
4.3 Development Environment.....	9
5.Solution Cofiguration	10
5.1 Application Setup.....	10
5.2 Issue DataTable Configuration	12
5.3 Employee Issue Record Producer	14
5.4 Record Producer Variables	17
5.5 Category and Subcategory Configuration	19
5.6 UI Policies	21
5.7 Catalog Client Script	24
5.8 Form Validation	27
5.9 Employee Center	28
5.10 Custom Widgets	30
5.11 End-to-End Workflow	31
5.12 Final Application Configuration	32
6. Testing and Output	33
6.1 Test Environment	34
6.2 Test Scenarios	34
6.3 Output Screenshots	35
7. Conclusion	36

1. SOLUTION CONTEXT

1.1 Business Context

In enterprise environments, employees may face different workplace concerns, including system or hardware problems, access issues, infrastructure-related requirements, payroll matters, and requests for HR clarification. A proper mechanism is required to collect these concerns in a consistent format and make the submitted information available for further processing.

When employees depend mainly on email messages or informal communication, important details may be missed or provided in different formats. This can create additional effort for support teams when reviewing, categorizing, and tracking employee concerns.

ServiceNow provides configurable platform features that can be used to build structured service-management solutions. These include Service Portal capabilities, Record Producers, custom tables, field controls, validation rules, and dependency configurations.

This project uses these ServiceNow capabilities to implement an Employee Issue Submission solution. Employees can access the issue-submission service through the portal and provide the required information using a Record Producer. The submitted information is then used to generate a corresponding record in the Employee Issue Custom Table.

The solution is designed to improve the consistency of submitted information through appropriate field controls, validation, and Category–Subcategory relationships.

1.2 Existing Challenge

Employees require an easy way to report workplace concerns without having to determine how the information should be stored or processed at the backend.

Using unstructured communication methods can result in the following challenges:

- Employee concerns may be submitted without a consistent format.
- Required information may be missing from the submission.
- Support teams may spend additional time organizing submitted requests.

- Issue classification can vary between submissions.
- Tracking the progress of employee concerns can become difficult.
- Manual handling may increase the effort required for support or HR teams.

These challenges create a requirement for a centralized submission mechanism where employees can enter issue details in a predefined format and where the information can be maintained as structured ServiceNow records.

1.3 Solution Approach

The project implements an **Employee Issue submission service using a ServiceNow Record Producer**.

Employees access the service through the Service Portal and enter the required issue information. The form is designed to collect important details such as:

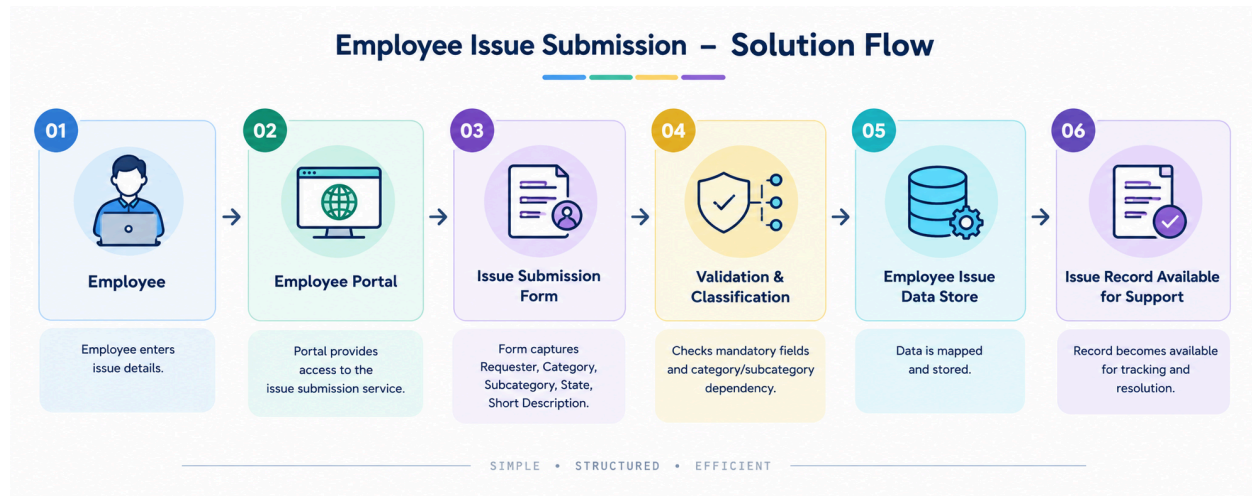
- Requester
- Category
- Subcategory
- State
- Short Description

Field behavior and validation rules are configured to guide the user during submission. The Category and Subcategory relationship also helps ensure that the selected classification values are related appropriately.

Once the information passes the configured checks, the Record Producer maps the submitted variables to the relevant fields of the **Employee Issue Custom Table**. A structured Employee Issue record is then generated in ServiceNow.

Process Flow

Employee → Service Portal → Employee Issue Form → Record Producer → Validation & Field Rules → Employee Issue Custom Table → Employee Issue Record



2. PROJECT GOALS

The project aims to provide a structured ServiceNow solution through which employees can submit workplace-related concerns using a simple and controlled interface. The solution captures the required information, applies configured rules, and creates a manageable issue record.

The major goals of the project are:

1. To provide employees with a convenient portal-based interface for submitting workplace concerns.
2. To design an Employee Issue submission process using a ServiceNow Record Producer.
3. To collect issue details through predefined and structured form variables.
4. To maintain submitted information in an Employee Issue Custom Table.
5. To classify employee concerns using suitable Category and Subcategory values.
6. To establish a dependency between Category and Subcategory selections.
7. To apply mandatory-field checks, UI Policies, and other configured validation rules.
8. To transfer the values entered by employees from the Record Producer to the appropriate issue fields.
9. To ensure that successful submissions result in the creation of a structured Employee Issue record.
10. To establish a configurable base that can support additional employee issue categories and future enhancements.

3. FUNCTIONAL HIGHLIGHTS

The solution provides the following major capabilities:

3.1 Employee Portal Entry

Employees can reach the issue-submission service from the configured ServiceNow portal and initiate a new workplace concern.

3.2 Guided Issue Submission

The Record Producer provides a dedicated form for entering issue information instead of giving employees direct access to the backend issue table.

3.3 Standardized Information Capture

The form collects predefined details such as:

- Requester
- Category
- Subcategory
- State
- Short Description

This provides a consistent structure for every submitted issue.

3.4 Dynamic Classification

The available Subcategory values are associated with the selected Category, helping the employee choose an appropriate issue classification.

3.5 Form Validation and Controls

Configured UI Policies and validation rules manage field visibility, mandatory conditions, read-only behavior, and other submission requirements.

3.6 Data Transfer to Backend

Information entered in the submission form is mapped to the relevant fields of the Employee Issue Custom Table.

3.7 Automated Issue Record Generation

After the form passes the configured checks and is submitted successfully, ServiceNow generates an Employee Issue record containing the submitted information.

3.8 Structured Issue Tracking

The generated records provide a consistent representation of employee concerns that can be used for subsequent review, tracking, and management.

This version keeps your actual project concept but changes the heading style, sentence structure, and presentation so it doesn't look like a direct copy of your friend's matter.

4. DEVELOPMENT ENVIRONMENT

4.1 Platform Access and Configuration

The project was developed using a ServiceNow development environment that provided the required platform features for building and validating the Employee Issue solution.

The development environment supported configuration activities such as:

- Custom table creation and field configuration
- Record Producer development
- Form variable setup
- UI Policy configuration
- Catalog Client Script configuration
- Service Portal setup
- Application testing and validation

These capabilities were used to build the employee issue submission workflow and verify its expected behavior.

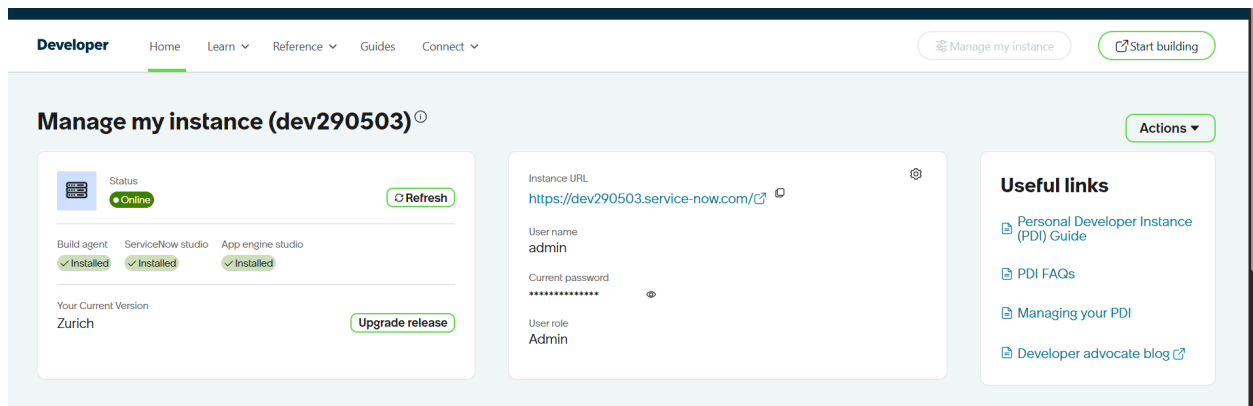
4.2 Development Instance

A Personal Developer Instance (PDI) was used as the working environment for implementing and testing the solution.

The PDI provided a separate environment where the project components could be configured and validated during development. The following activities were carried out as part of the implementation:

- Configuring the Employee Issue Custom Table
- Creating the Employee Issue submission Record Producer
- Defining the required form variables
- Setting up Category and Subcategory values
- Applying UI Policies and field validation rules
- Establishing Category–Subcategory dependency behavior
- Testing the creation of Employee Issue records
- Verifying the employee-facing portal submission flow

This development setup supported the configuration, testing, and validation of the complete Employee Issue submission process.



4.3 Development Environment

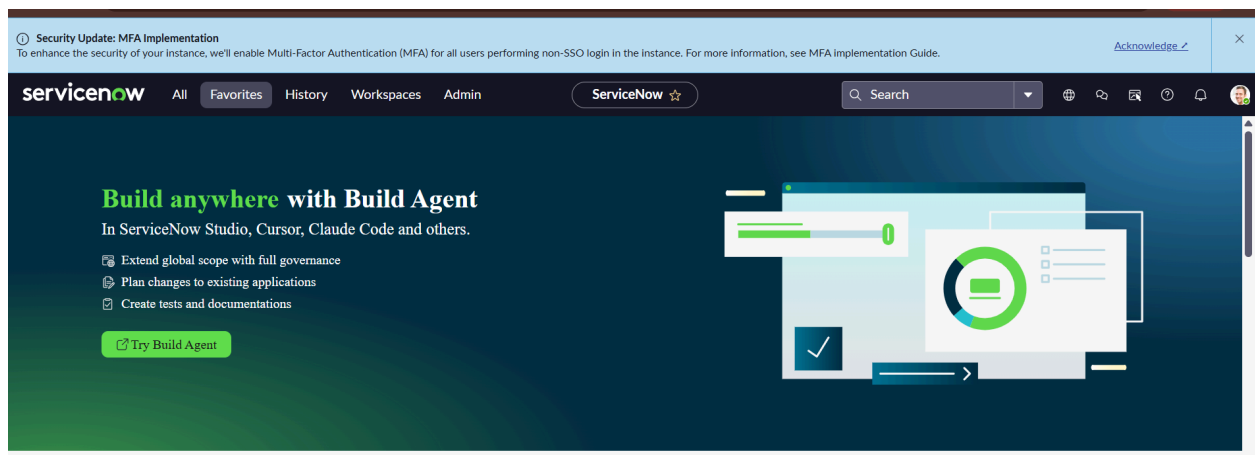
The project was implemented and tested using the built-in development and configuration features of the ServiceNow platform. The main components used were Custom Tables, Record Producers, Variables, UI Policies, Catalog Client Scripts, Service Portal configuration, and validation controls. These components were configured and tested to support the Employee Issue submission process.

4.3 Development Components

Component	Role in the Project
ServiceNow Platform	Used for application configuration and development
Employee Issue Custom Table	Maintains submitted workplace issue records

Record Producer	Provides the form for submitting employee issues
Record Producer Variables	Collects the required issue details
UI Policies	Manages field visibility, mandatory conditions, and behavior
Catalog Client Script	Supports dynamic Category–Subcategory behavior
Employee Center	Provides employees access to the issue submission service
ServiceNow Data Layer	Stores and retrieves the generated issue records

These components collectively support the process from employee input through validation and submission to Employee Issue record creation.



End-to-End Development Flow

Service Portal → Record Producer → Variables → Validation & Dependency Controls → Employee Issue Custom Table → Employee Issue Record

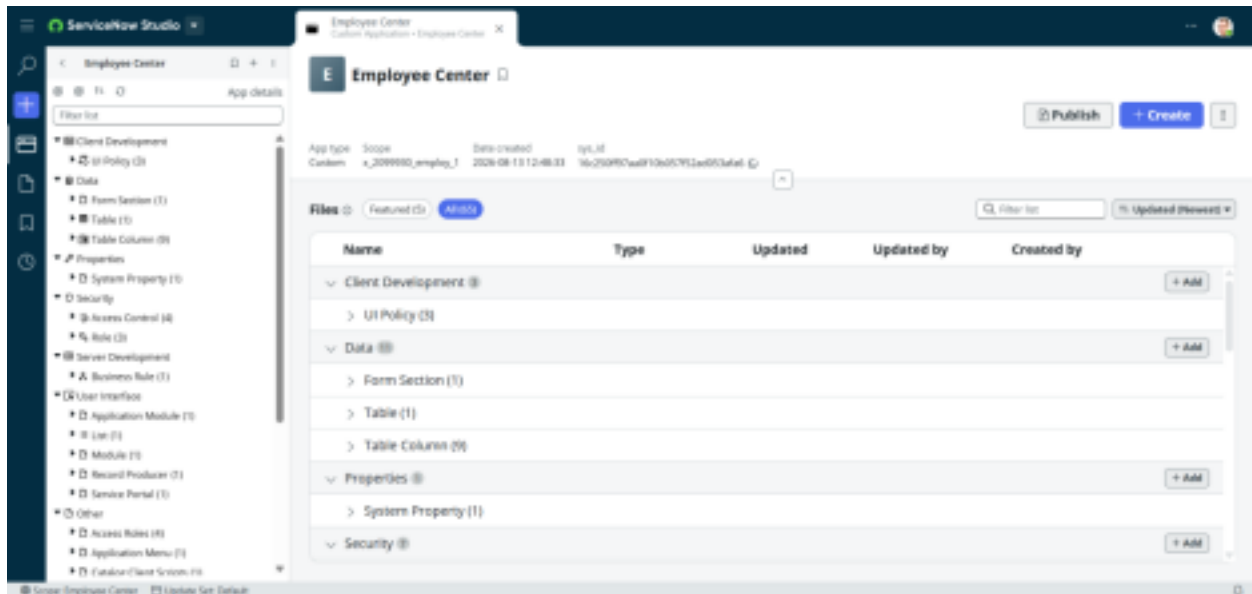
5. SOLUTION CONFIGURATION

5.1 Application Setup

The Employee Issue solution was configured within the ServiceNow development environment. The required application components were organized and maintained

within the application scope, including the Employee Issue Custom Table and its related configuration elements.

This setup provided the base for developing the employee issue submission process and connecting the required ServiceNow components.



5.1 Application Structure

The Employee Issue solution was organized within a dedicated ServiceNow application environment. This application provides the base structure for configuring the data model, employee-facing submission form, and supporting validation components.

The application setup brings together the major elements required for the solution, including the Employee Issue Custom Table, Record Producer, form variables, UI Policies, validation rules, Category–Subcategory relationship, and portal access.

Using an application-based structure keeps the project configuration organized and provides a suitable foundation for future enhancements.

Application Scope

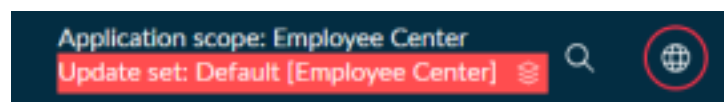
The application scope provides a dedicated area for maintaining the configuration developed for the Employee Issue solution. The related tables, submission components, and form controls are configured within the project environment, helping keep the solution components organized and easier to maintain.

Configuration	Project Details
Application Category	Custom ServiceNow Application
Purpose	Manage employee workplace issue submissions
Data Structure	Employee Issue Custom Table
Employee Access	ServiceNow Employee Center / Service Portal
Submission Method	Employee Issue Record Producer
Supporting Components	Variables, UI Policies, Validation Rules, Client-side Configuration

This application structure forms the base for the subsequent configuration of the Employee Issue Custom Table and employee issue submission workflow.

Application Scope

The application scope provides a dedicated space for the project configuration in ServiceNow. The Employee Issue components are maintained within this scope, keeping the related tables, forms, and configuration elements organized as part of the overall solution.



Implementation Outcome

The initial application setup was completed successfully and provided the required foundation for developing the Employee Issue solution. The configured application environment supports the project-specific data structure and the ServiceNow components required for employee issue submission.

The next stage of implementation focuses on setting up the Employee Issue Custom Table, which is used to maintain the information submitted through the employee issue form.

5.2 Issue Data Table Configuration

The Employee Issue Custom Table was configured to maintain the information entered by employees during the issue submission process. It provides the required data structure for storing and managing the workplace concerns submitted through the Record Producer.

The table is maintained within the configured application environment and serves as the main data object for the Employee Issue solution.



Field	Purpose
Number	Unique issue reference
Requester	Employee who raised the issue
category	Main issue classification
Subcategory	Detailed issue type
Short Description	Brief issue summary
State	Current issue status
Priority	Issue importance level
Assignment Group	Responsible support team
Created / Updated	Record timestamps
Created By / Updated By	User details
Sys ID	Unique system identifier
Updates	Number of record changes

Columns Controls Application Access						
Table Columns: Column label Search						
Dictionary Entries						
	Column label	Type	Reference	Max length	Default value	Display
X	Assignment Group	Reference	Group	32		false
X	Category	String	(empty)	40		false
	Created	Date/Time	(empty)	40		false
	Created by	String	(empty)	40		false
X	Number	String	(empty)	40	javascript:global.getNextObj(NumberOfFieldID);	false
X	Priority	String	(empty)	40		false
X	Requester	Reference	User	32		false
X	Short Description	String	(empty)	40		false
X	State	String	(empty)	40		false
X	Subcategory	String	(empty)	40		false
	Sys ID	Sys ID (GUID)	(empty)	32		false
	Updated	Date/Time	(empty)	40		false
	Updated by	String	(empty)	40		false
	Updates	Integer	(empty)	40		false
	Insert a new row...					

Table Form Setup

The Employee Issue form is designed to display and manage submitted issue records. It presents important details such as Requester, Category, Subcategory, Short Description, State, Priority, and Assignment Group in an organized format.

Employee custom issue
EMP00000015

Number

EMP00000015

Requester

Fred Luddy

Category

Hardware

Subcategory

Laptop

Short Description

Test Case TC-006

State

New

Priority

Medium

Assignment Group

Hardware

Update

Delete

[Open URL in Windows Explorer](#)

Data Storage and Record Creation

The Employee Issue Custom Table acts as the main storage area for submitted workplace issues. After form submission, the entered values are transferred to the relevant fields and a new issue record is generated.

Implementation Outcome

The table was configured successfully to store and maintain employee issue information. It supports further activities such as tracking, assignment, priority handling, and issue management.

5.3 Employee Issue Record Producer

The Employee Issue Record Producer provides the form through which employees submit workplace concerns from the ServiceNow portal. It offers a controlled interface

without requiring employees to access the backend table directly.

When the form is submitted successfully, the entered information is used to create a new record in the Employee Issue Custom Table.

The screenshot shows the 'Record Producer' configuration for 'Raise Employee Issue'. The interface includes a header with navigation buttons (Copy, Try It, Update, Edit in Catalog Builder, Delete) and a sub-header 'Build and modify items faster with the improved Catalog Builder'. The main configuration area has fields for Name (Raise Employee Issue), Application (Employee Center), Table name (Employee raises issue (s_3099938).empl...), State (None), Checked out (None), and Owner (System Administrator). Below this, there are tabs for 'What it will contain', 'Accessibility', 'Generated Record Data', and 'Portal Settings'. The 'What it will contain' tab is active, showing a 'Catalog' dropdown set to 'Service Catalog', a 'Category' dropdown set to 'Services', and a 'View' dropdown. A 'Can cancel' checkbox is also present. At the bottom, there are buttons for Copy, Try It, Update, Edit in Catalog Builder, and Delete.

Implementation Result

The Raise Employee Issue Record Producer was successfully created and integrated with the Employee Issue custom table. It is designed to collect the necessary issue-related details from employees and automatically transfer the submitted information to the appropriate fields in the Employee Issue record.

5.4 Record Producer Variables

The Raise Employee Issue Record Producer contains a set of variables that gather the required information from employees while submitting an issue. These variables appear as input fields on the employee-facing form. Each variable is configured to capture specific issue details and map the entered values to the respective fields in the Employee Issue custom table.

The screenshot shows the 'Raise Employee Issue' form. The header includes a menu icon, the title 'Raise Employee Issue', and the instruction 'Please provide the details below to raise your issue.' The form contains several input fields: 'What is the Requester's Name?' with a text input field labeled 'Enter your name'; 'Which Category?' with a dropdown menu labeled 'Select category'; 'What is Subcategory?' with a dropdown menu labeled 'Select subcategory'; 'Short Description' with a text area labeled 'Enter a brief description of the issue'; and 'Which State is this issue in?' with a dropdown menu labeled 'Select state'.

Configured Variables

Variable	Purpose
Requester	Identifies the employee submitting the issue request.
Category	Defines the general classification of the reported issue.
Subcategory	Specifies the particular issue type under the selected category.
State	Indicates the current state or status related to the submitted issue.
Short Description	Provides a brief summary of the issue reported by the employee.

These variables create a structured input mechanism for collecting employee issue details. Instead of exposing the fields of the underlying Employee Issue custom table directly, employees provide the required information through the Record Producer form.

The Requester variable records the employee associated with the request. Category and Subcategory help organize and identify the type of issue. Short Description records a brief explanation of the problem, while State stores the relevant status information for the submitted issue.

Variable-to-Field Relationship

The variables configured in the Record Producer act as the input layer for the Employee Issue custom table. When an employee submits the form, the entered values are transferred by the Record Producer and stored in their respective fields of the target table. This ensures that the submitted issue information is captured consistently and creates a new Employee Issue record with the required details.

Question	Field
What is the Requester's Name?	u_requester
Which Category?	u_category
What is Subcategory?	u_subcategory
Short Description	u_short_description
Which State is this issue in?	u_state

Variable Mapping and Data Flow

The overall data flow of the Record Producer can be described as:

Employee Submission → Record Producer Variables → Validation and Dependency Rules → Employee Issue Table Fields → Employee Issue Record

This approach creates a clear separation between the form used by employees and the database structure behind the application. The Record Producer collects the required information through variables, applies the configured controls, and transfers the validated values to the appropriate fields in the Employee Issue table.

Implementation Result

The Record Producer variables were configured successfully to collect the information required for submitting an employee issue. These variables act as the initial data-entry layer of the Raise Employee Issue process and supply the necessary values for generating a properly structured Employee Issue record.

5.5 Category and Subcategory Configuration

The Category and Subcategory fields are used to organize employee issues in a systematic manner. The Category represents the general area or type of the reported issue, whereas the Subcategory provides a more detailed classification within the selected Category.

Using this two-level classification makes issue records more consistent and helps in managing and processing employee requests efficiently. The application is configured so that the available Subcategory choices depend on the Category selected by the employee.

Category Configuration

The **Category** variable is created using a Select Box and is associated with the **u_category** field in the Employee Issue custom table. While submitting an issue through the Raise Employee Issue Record Producer, the employee selects the category that best represents the reported problem.

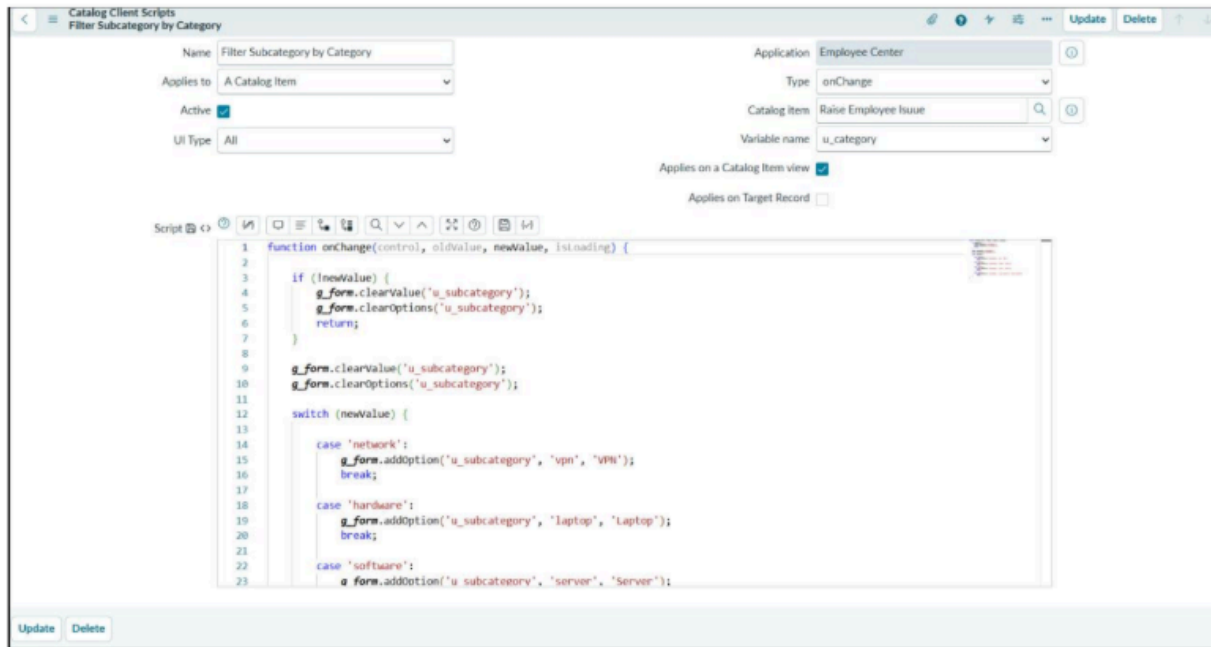
Subcategory Configuration

The Subcategory variable is also configured as a Select Box and is linked to the u_subcategory field of the Employee Issue custom table. Its available choices are dynamically controlled based on the Category selected by the employee, ensuring that only relevant subcategory options are presented.

Category–Subcategory Dependency

A dependency is established between the Category and Subcategory variables to ensure that the employee selects a relevant Subcategory for the chosen Category. Once a Category is selected, the system filters or controls the available Subcategory choices based on the configured relationship.

This dependency helps avoid incorrect or unrelated classifications and ensures that Employee Issue records contain consistent and meaningful category information. It also makes the issue submission process easier by displaying only the Subcategory options that are applicable to the selected Category.



Implementation Result

The **Category–Subcategory** setup provides an organized method for classifying Employee Issues. The relationship configured between these two fields ensures that the available Subcategory values are based on the Category selected by the employee. This improves data consistency and minimizes the chances of assigning an inappropriate classification.

The configured dependency was also validated during project testing. **Test Case TC-003** confirms that the appropriate Subcategory choices are presented when different Categories are selected, and the observed results matched the expected behavior.

5.6 UI Policies

UI Policies were implemented on the Employee Issue table to dynamically control the behavior of form fields. These policies help protect automatically generated information and ensure that important fields follow the required conditions during the issue lifecycle, particularly when an issue is in the **New** state.

5.6.1 Number Field Read-Only Policy

A UI Policy titled “**Number field will read only**” was configured for the Employee Issue

table.

A corresponding **UI Policy Action** was added for the **Number** field with **Read only = True**. Since the policy does not contain any specific condition, the Number field remains non-editable whenever the Employee Issue form is displayed. This prevents users from manually modifying the system-generated issue number.

The screenshot shows the configuration for a UI Policy titled "Number field will read only." The policy is associated with the "Employee raises issue [x_2099930_emp]" table and the "Employee Center" application. It is currently active. The "When to Apply" section is empty, indicating no conditions are defined. Below this, there are links for "Update", "Delete", "Convert this to Data Policy", "Advanced view", "Run Point Scan", and "ISN [URL] Versions (1)". The "UI Policy Actions (1)" tab is selected, showing a table of actions.

Field name	Mandatory	Visible	Read only
number	Leave alone	Leave alone	True

Purpose: To ensure that the system-generated Employee Issue number cannot be edited manually by users.

5.6.2 State Field Read-Only Policy

A UI Policy titled "**State field will be read only**" was configured on the **Employee Issue** table.

The policy includes a **UI Policy Action** for the **State (u_state)** field, where **Read only = True** is enabled. Since no condition is defined for this policy, the State field remains non-editable on the Employee Issue form. This prevents users from directly changing the state value through the form.

UI Policy: State field will be read only.

Table: Employee raises issue [x_2099930_empl...]

Application: Employee Center

Active: ☒

* Short description: State field will be read only.

When to Apply

Conditions: Add Filter Condition Add OR Clause

-- choose field -- -- oper -- -- value --

Update Delete

Related Links

[Convert this to Data Policy](#)

[Advanced view](#)

[Run Point Scan](#)

[\[SN Units\] Versions \(1\)](#)

UI Policy Actions (1)

UI Policy Related List Actions

UI policy = State field will be read only.

Field name	Mandatory	Visible	Read only
u_state	Leave alone	Leave alone	True

Purpose: To make the **Short Description** mandatory when an Employee Issue is in the **New** state, ensuring that the issue contains the basic information required for further processing.

5.6.3 Summary of UI Policies

UI Policy	Condition	Field/Action	Result
Number field will be read only	None	Number → Read only	The Number field cannot be modified by the user
State field will be read only	None	State → Read only	The State field remains non-editable.
When the state changes to New, Short Description is mandatory	State is New	Short Description → Mandatory	A description must be entered when the issue is in the New state.

These UI Policies regulate the behavior of important fields on the Employee Issue form. They protect system-controlled values and ensure that the necessary issue information is available before the record is processed further.

5.7 Catalog Client Script

A **Catalog Client Script** was implemented to manage the **Category–Subcategory relationship** within the **Raise Employee Issue Record Producer**. The script dynamically updates the available Subcategory choices according to the Category

selected by the employee.

5.7.1 Catalog Client Script

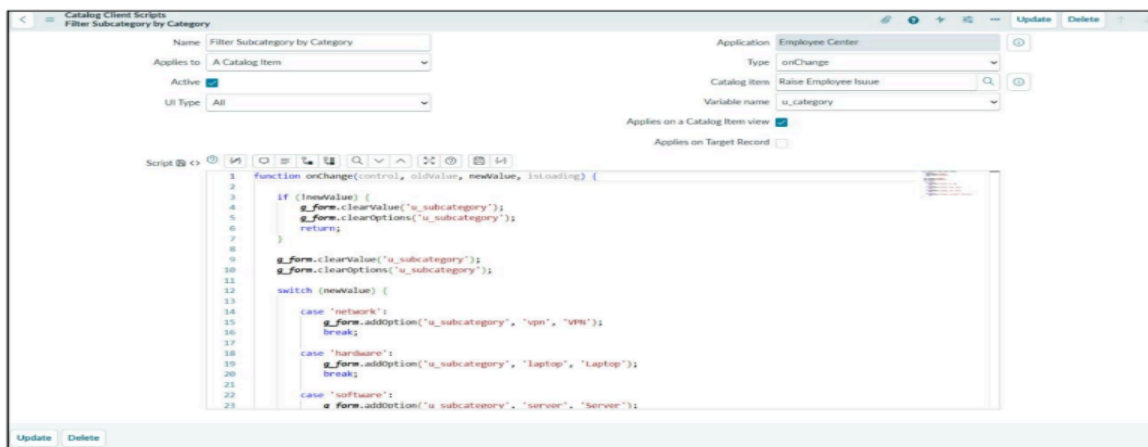
A Catalog Client Script named **“Filter Subcategory by Category”** was created for the **Raise Employee Issue Record Producer**.

The script runs whenever the **Category** variable is changed. On each change, the existing Subcategory value is cleared and the previously displayed choices are removed. The script then loads the Subcategory option that corresponds to the newly selected Category.

Category	SubCategory
Network	VPN
Hardware	Laptop
Software	Server
Access	Forgot Password

The configured mapping is: This configuration keeps the Subcategory values aligned with the selected Category and prevents users from choosing options that are unrelated to the issue type.

This configuration keeps the Subcategory choices relevant to the selected Category and avoids displaying inappropriate or unrelated options to the employee.



5.7.2 Script Logic

The Catalog Client Script follows a defined sequence to update the Subcategory values dynamically:

1. Determine whether the script is being executed during the initial form load.
2. If the form is still loading, stop the execution of the change-based logic.
3. Remove the value currently selected in the **Subcategory** field.
4. Clear the existing Subcategory choices from the variable.
5. Identify the Category selected by the employee.
6. Insert the corresponding Subcategory option based on the selected Category.

The configured Category–Subcategory relationship is shown below:

Category	SubCategory
Network	VPN
Hardware	Laptop
Software	Server
Access	Forgot Password

With this logic, employees can select only the Subcategory that corresponds to the chosen Category.

5.7.3 Dependency Behavior

The Category–Subcategory relationship operates dynamically on the employee-facing **Raise Employee Issue Record Producer** form.

For example:

- **Category = Hardware** → Available Subcategory: **Laptop**
- **Category = Software** → Available Subcategory: **Server**

When the employee selects a different Category, the previously selected Subcategory is automatically removed and the available choice is refreshed based on the new Category. This helps avoid mismatched selections and keeps Employee Issue classification accurate.

5.7.4 Initial-Load Handling

During the testing phase, an issue was noticed during the initial loading of the Record

Producer. When the form opened with **Network** already selected, unrelated Subcategory options could briefly appear. However, the filtering function worked correctly after the Category value was changed.

To resolve this behavior, the Catalog Client Script was updated to properly handle the form-loading condition. Following this adjustment, the Subcategory list was correctly initialized and unrelated options were no longer displayed during the initial form load.

Implementation Result

The “**Filter Subcategory by Category**” **Catalog Client Script** was successfully implemented in the **Raise Employee Issue Record Producer**. It dynamically manages the Subcategory choices based on the selected Category, helping maintain accurate and consistent Employee Issue classification.

5.8 Form Validation

The **Employee Raise Issue** process includes several validation and field-control mechanisms to ensure that submitted information follows the application's requirements.

The validation behavior is achieved through the configured **UI Policies** and **Catalog Client Script**. UI Policies manage the behavior of fields on the Employee Issue form, while the Catalog Client Script maintains the Category–Subcategory dependency within the Record Producer.

The implemented controls provide the following behavior:

- **Short Description** becomes mandatory when the issue State is **New**.
- The **Number** field remains read-only on the Employee Issue form.
- The **State** field remains read-only on the Employee Issue form.
- The available **Subcategory** values depend on the selected Category.
- When the Category is changed, the existing Subcategory value is cleared and the relevant option is loaded.

Validation Flow

The complete validation and submission process can be represented as:

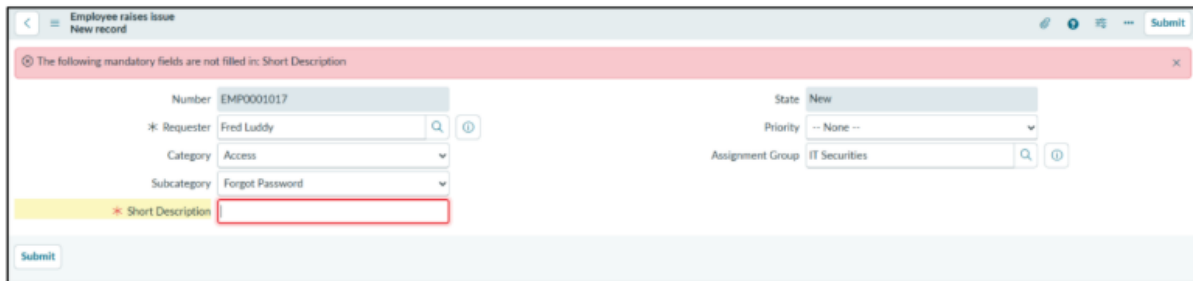
Employee Input → Record Producer Validation → Category–Subcategory Check → Record Producer Processing → Employee Issue Record Creation

After all configured requirements are satisfied, the Record Producer processes the submitted information and creates the corresponding **Employee Issue** record.

For example, during testing, an attempt to submit the form without entering a **Short Description** while the State was **New** was blocked. This confirmed that the mandatory-field validation was functioning correctly.

Validation Result

The configured **UI Policies** and **Catalog Client Script** were verified during testing of the Employee Raise Issue process. The UI Policies successfully enforce the required field behavior on the Employee Issue form, while the Catalog Client Script correctly manages the Category–Subcategory relationship within the Record Producer.



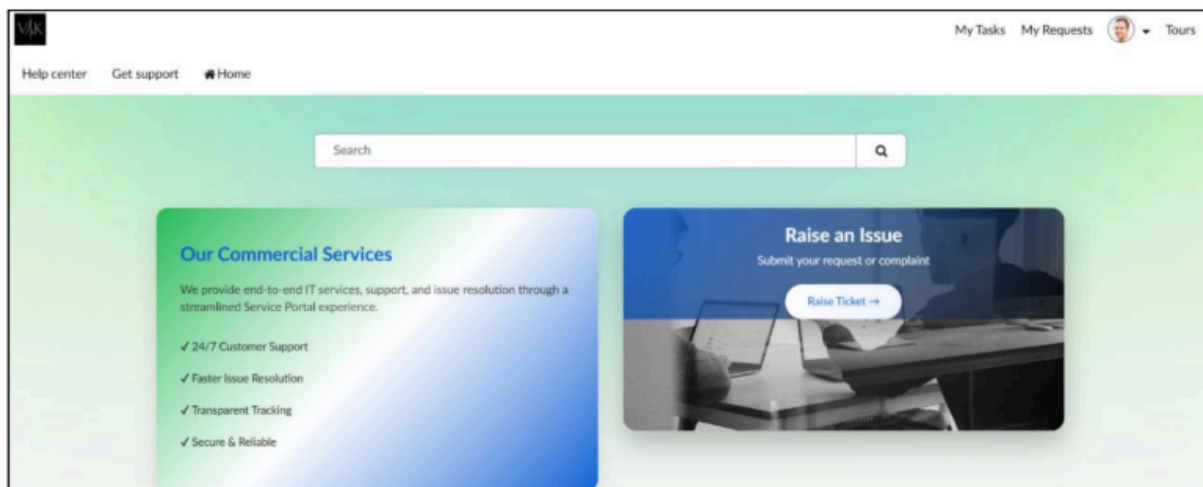
The screenshot shows a web form titled "Employee raises Issue" with a sub-header "New record". A red error banner at the top states: "The following mandatory fields are not filled in: Short Description". The form contains several fields: "Number" (EMP0001017), "Requester" (Fred Luddy), "Category" (Access), "Subcategory" (Forgot Password), "State" (New), "Priority" (None), and "Assignment Group" (IT Securities). The "Short Description" field is highlighted with a red border and a red asterisk, indicating it is a mandatory field that has not been filled. A "Submit" button is located at the bottom left of the form.

5.9 Employee Center

The Employee Center acts as the employee-facing portal through which users can access the issue-raising functionality. The application was configured to make the Raise Employee Issue Record Producer easily accessible from the portal using multiple entry methods.

5.9.1 Access through the Raise an Issue Widget

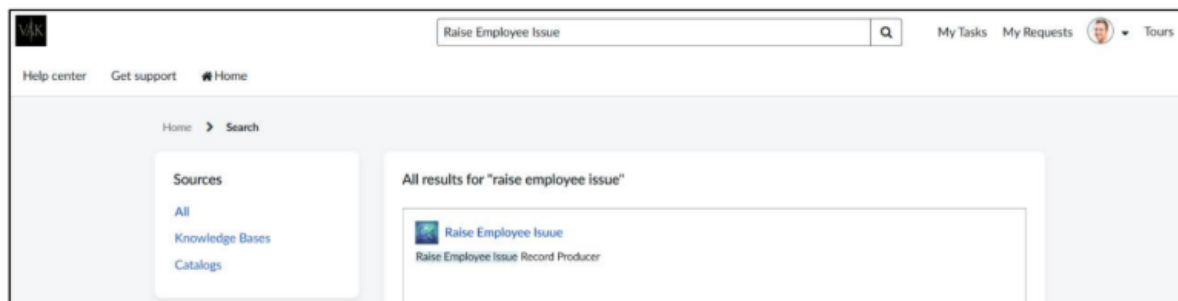
A customized Raise an Issue widget was added to the Employee Center home page. The widget provides a dedicated Raise Ticket option that employees can select to begin submitting a new issue. This provides a simple and direct way for employees to access the issue submission form from the portal.



Selecting **Raise Ticket** takes the employee directly to the **Raise Employee Issue Record Producer**. This provides a straightforward way to start a new issue submission while minimizing unnecessary navigation steps.

5.9.2 Access through Portal Search

Employees can also locate the **Raise Employee Issue Record Producer** using the search option available in the Employee Center. By entering “**Raise Employee Issue**” in the portal search bar, the related Record Producer is displayed in the search results, allowing the employee to open the submission form directly.



This search-based option gives employees another convenient way to find the required service when they prefer using the portal search instead of the dedicated widget.

5.9.3 Employee Access Flow

The available access methods for the issue submission functionality can be summarized as follows:

Employee Center Home Page → Raise an Issue Widget → Raise Ticket → Raise Employee Issue Record Producer

OR

Employee Center Search → “Raise Employee Issue” → Record Producer

Regardless of the selected access method, both routes direct the employee to the same Raise Employee Issue Record Producer. Once the form is opened, the configured variables, Category–Subcategory dependency, validation controls, and other application configurations are applied during the submission process.

5.10 Custom Widgets

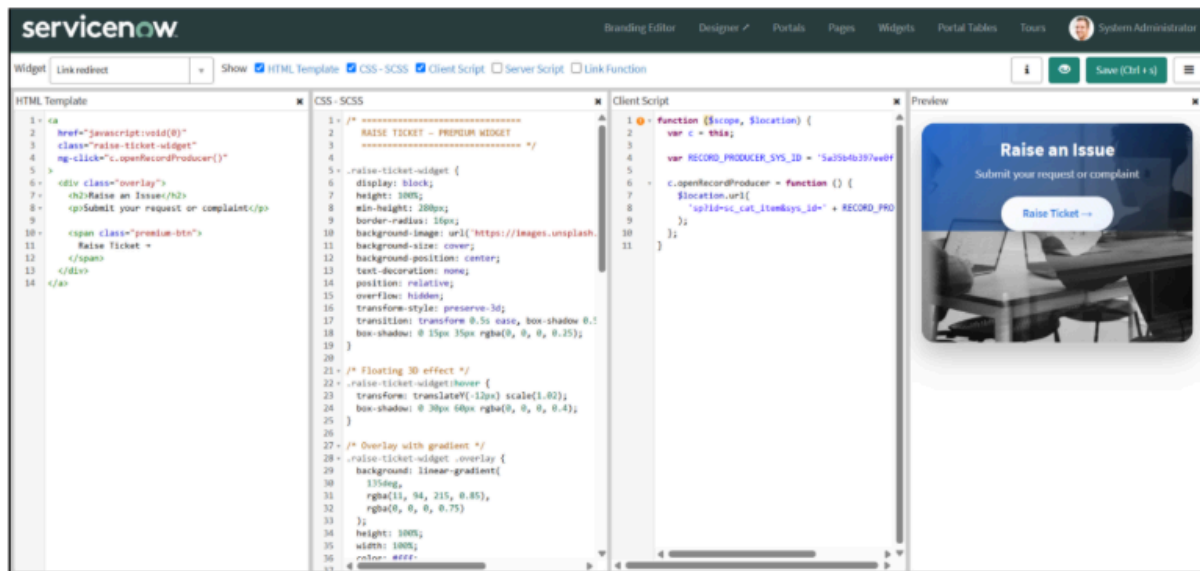
A customized Raise an Issue widget is available on the Employee Center home page. It serves as a convenient starting point for employees who need to submit a new Employee Issue.

5.10.1 Raise an Issue Widget

The custom widget is placed directly on the Employee Center landing page and includes a dedicated **Raise Ticket** option.

The main purpose of this widget is to give employees a simple and visible way to start an issue request. Instead of navigating through multiple portal pages or manually searching for the Record Producer, employees can use the widget to access the Raise Employee Issue submission form directly.

Figure 5.10.1



5.10.2 Widget Behavior

When an employee clicks the **Raise Ticket** option, the custom widget navigates the user directly to the **Raise Employee Issue Record Producer**.

The navigation sequence is:

Employee Center → Raise an Issue Widget → Raise Ticket → Raise Employee Issue Record Producer

After opening the Record Producer, the employee is presented with the configured input variables. The Category–Subcategory relationship and the defined validation controls are then applied during the submission process.

5.10.3 Widget Implementation Result

The **Raise an Issue** widget was successfully added to the **Employee Center** home page and connected to the Employee Issue submission process. It provides employees with a visible and convenient entry point, reducing the amount of navigation needed to

create a new issue.

5.11 End-to-End Workflow

The **Employee Raise Issue** application brings together the different ServiceNow configurations into one complete issue-reporting process.

The workflow starts when an employee enters the Employee Center and ends after the submitted information is converted into an **Employee Issue** record in the custom table.

5.11.1 Complete Workflow

The complete process can be represented as:

Employee → Employee Center → Raise an Issue Widget / Portal Search → Raise Employee Issue Record Producer → Record Producer Variables → Validation & Dependency Controls → Employee Issue Custom Table → Employee Issue Record

The employee can enter the issue submission process either through the **Raise an Issue** widget or by searching for **Raise Employee Issue** in the Employee Center. Both options open the same Record Producer.

The employee then enters the required information using the configured variables:

- **Requester**
- **Category**
- **Subcategory**
- **State**
- **Short Description**

The **Category–Subcategory dependency** dynamically controls the available Subcategory based on the Category selected by the employee.

The configured **UI Policies** manage field requirements and read-only restrictions on the Employee Issue form. Once the necessary information satisfies the configured controls, the Record Producer processes the submission and creates a new record in the **Employee Issue custom table**.

5.11.2 Workflow Representation

Employee



Employee Center



Raise an Issue Widget / Portal Search



Raise Employee Issue Record Producer



Record Producer Variables



Validation & Category–Subcategory Dependency



Employee Issue Custom Table



Employee Issue Record

5.11.3 Implementation Result

The complete workflow successfully links the employee portal with the Record Producer, validation mechanisms, custom table, and final Employee Issue record.

All configured components operate together to provide a structured and consistent process for submitting and storing employee issues.

5.12 Final Application Configuration

After implementing the individual features, the **Employee Raise Issue** application was reviewed as an integrated ServiceNow solution to verify that all components work together correctly.

5.12.1 Application Components

Component	Configuration
Application	Employee Center
Main Data Object	Employee Issue custom table
Table Name	x_2099930_employ_1_employee_raises_issue
Submission Mechanism	Raise Employee Issue Record Producer
Employee Interface	Employee Center
Input Layer	Record Producer Variables
Classification	Category and Subcategory

Dependency	Category–Subcategory filtering
Field Controls	UI Policies
Client-Side Logic	Catalog Client Script
Portal Access	Raise an Issue widget and portal search

The above configuration brings together the application, Employee Issue table, Record Producer, variables, UI Policies, client-side dependency logic, and Employee Center features described in the earlier sections.

5.12.2 Final Data Flow

The complete data movement within the application can be represented as:

Employee Input → Record Producer Variables → Validation & Dependency Controls → Employee Issue Table Fields → Employee Issue Record

This flow ensures that information entered by the employee is processed through the configured controls before being stored as a structured Employee Issue record.

5.12.3 Final Implementation Result

The Employee Raise Issue application was successfully configured with the required ServiceNow components and supporting functionality.

The completed solution provides:

- Employee-facing issue submission through the Employee Center.
- Record creation using a Record Producer.
- Structured collection of employee issue details.
- Category–Subcategory dependent classification.
- Field validation and read-only controls through UI Policies.
- Dynamic client-side filtering using a Catalog Client Script.
- Storage of submitted information in the Employee Issue custom table.
- Multiple ways for employees to access the issue submission form through the Employee Center.

6. TESTING AND OUTPUT

The completed **Employee Raise Issue** application was tested in the **ServiceNow Personal Developer Instance (PDI)** to confirm that the configured features were functioning as expected. Testing covered employee access, form submission, field

validation, Category–Subcategory dependency, portal navigation, and successful creation of Employee Issue records.

6.1 Test Environment

All testing activities were carried out in the same **ServiceNow Personal Developer Instance** used during application development.

The following application components were considered during testing:

- **Employee Center**
- **Raise Employee Issue Record Producer**
- **Employee Issue custom table**
- **Record Producer variables**
- **UI Policies**
- **Category–Subcategory dependency**
- **Catalog Client Script**

The PDI provided a controlled environment for verifying the application configuration and observing the behavior of the implemented features.

6.2 Test Scenarios

Test Case	Test Condition	Expected Result	Result
TC-001	Access the Raise Employee Issue option through the Employee Center widget	The Raise Employee Issue Record Producer should open successfully	Passed
TC-002	Search for Raise Employee Issue using Employee Center search	The corresponding Record Producer should be displayed in the search results	Passed
TC-003	Select a Category	The relevant Subcategory option should be displayed	Passed
TC-004	Change the selected Category	The previously selected Subcategory should be cleared and the applicable option should be updated	Passed

TC-005	Keep State as New and leave Short Description empty	The form submission should be prevented	Passed
TC-006	Enter valid issue details and submit the form	A new Employee Issue record should be created	Passed
TC-007	Open the newly created Employee Issue record	The submitted information should appear in the corresponding fields	Passed

The testing confirmed that the Category–Subcategory dependency worked correctly, which was documented as **TC-003**. The validation behavior was also verified by testing the condition where **State is New** and Short Description is left empty. In this case, the application correctly prevented the submission.

6.3 Output Screenshots

The major outputs of the implemented application are shown through the screenshots included in the implementation sections.

Employee Center Access

Figure 21 shows the Employee Center home page with the **Raise an Issue** widget and its **Raise Ticket** option. This provides employees with a direct way to access the issue submission form.

Portal Search

Figure 22 shows the result of searching for **Raise Employee Issue** through the Employee Center search functionality. The related Record Producer is displayed in the search results.

Record Producer

Figure 10 presents the employee-facing **Raise Employee Issue** form containing the configured input variables used for submitting an issue.

Validation

Figure 20 demonstrates the validation behavior for the **Short Description** field when the **State** is set to **New**.

Created Employee Issue Record

Figure 8 displays the Employee Issue record generated after successful submission, including the information entered by the employee.

Together, these outputs verify the complete application behavior, including employee access, issue submission, validation, dynamic field handling, and creation of the final Employee Issue record.

7. CONCLUSION

The **Employee Raise Issue Management** application was successfully developed and configured using the capabilities provided by the ServiceNow platform.

The application offers employees a centralized method for reporting workplace-related issues through the Employee Center. The Raise Employee Issue Record Producer collects the required issue details and uses the submitted information to create a corresponding record in the Employee Issue custom table.

The solution incorporates several ServiceNow features, including **Category–Subcategory dependency, UI Policies, Catalog Client Script, form validation, Record Producer variables, and multiple Employee Center access methods**. These components work together to provide a structured process for capturing and storing employee issues.

The completed workflow connects the employee-facing portal with the Record Producer, validation and dependency controls, custom table, and final Employee Issue record. This demonstrates how different ServiceNow configuration elements can be integrated to build an organized employee-service application.

Overall, the project provides a functional foundation for managing employee issue submissions within ServiceNow. The application can be further enhanced in the future by adding features such as assignment management, email notifications, workflow automation, reporting, dashboards, and other employee-service capabilities.